OS ORANGE PROBLEM 4

NAME: ATHARVA MIHIR GORE

SRN: PES1UG24AM346

SECTION: AIML F

PHASE 1

```
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$
```

```
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$
```

PHASE 2

```
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$
```

PHASE 3

```
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ echo "hello" > file1.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ echo "world" > file2.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes add file1.txt file2.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  README.md
  untracked:  pes.c
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  test_tree
  untracked:  test_objects.c
  untracked:  index.h
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
  untracked:  object.c
  untracked:  .gitignore
  untracked:  commit.h
  untracked:  index.c
  untracked:  commit.c
  untracked:  test_objects
  untracked:  pes.h
```

```
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776678191 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776678200 6 file2.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$
```

PHASE 4

```
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ echo "Hello" > hello.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes add hello.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes commit -m "add world"
Committed: 0ae71511fd8d... add world
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ echo "world" >> hello.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes add hello.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes commit -m "add world (fr)"
Committed: c97289eb8d1e... add world (fr)
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ echo "Goodbye" > bye.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes add bye.txt
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes commit -m "add farewell"
Committed: 88955779faae... add farewell
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ ./pes log
commit 88955779faae360ffcf7fa70ba59a5870ecab4c44c3f5c57b86d2c5511e91ce1
Author: PES User <pes@localhost>
Date:   1776678843

    add farewell

commit c97289eb8d1e8c44548d2acf2c53d14d343192e2bb14b6012e9b818d5312101b
Author: PES User <pes@localhost>
Date:   1776678795

    add world (fr)

commit 0ae71511fd8d25df94b1e0374351869f75b5bf14622f5aee9fff67ab1fe5b746
Author: PES User <pes@localhost>
Date:   1776678720

    add world

add worlder <Q
```

```
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/0a/e71511fd8d25df94b1e0374351869f75b5bf14622f5aee9fff67ab1fe5b746
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/88/955779faae360ffcf7fa70ba59a5870ecab4c44c3f5c57b86d2c5511e91ce1
.pes/objects/9d/6f801b80330e5291d2128c6e03e6768733c50eeddc90d6207be19e16e8ed11
.pes/objects/b8/4080f7d65f2be4e6cedcc3f27db0273a213059d60df8ca47189920c3d45ba7
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/be/03a393adb03a84878402ea8937c2af2b745821c23f413865d2f36fbf2e2515
.pes/objects/c9/7289eb8d1e8c44548d2acf2c53d14d343192e2bb14b6012e9b818d5312101b
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$
```

```
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ cat .pes/refs/heads/main
88955779faae360ffcf7fa70ba59a5870ecab4c44c3f5c57b86d2c5511e91ce1
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
atharva@linux:~/programs/c/os/PES1UG24AM346-pes-vcs$
```

Q5.1

Ans: The HEAD file is updated to contain the reference to the new branch (e.g., ref: refs/heads/<branch>).
2. The .pes/index file must be rewritten to match the exact tree structure of the commit that the target branch points to.

It must be synchronized with the target branch's tree. Files that exist in the target branch but not in the current working directory must be created. Files that exist currently but not in the target branch must be deleted. Files that exist in both but differ must be overwritten with the target branch's version.
The primary complexity lies in preventing data loss. If the user has modified files in their working directory but hasn't committed them, a naive checkout would blindly overwrite their hard work. The checkout algorithm must carefully diff the working directory, the index, the current HEAD commit, and the target branch commit to determine if it is safe to switch.

Q5.2

Ans:

> Read the index: For a given tracked file, hash the current physical file in the working directory and compare it against the hash stored in the .pes/index. If they differ, the file is "dirty" (locally modified).Check the target commit: Look up that same file's path in the target branch's root tree (recursively traversing tree objects in the store). Find its expected hash.Evaluate conflict: If the file is dirty and the target branch expects a different hash for that file compared to the current HEAD's tree, a conflict exists. The checkout must refuse to run, because switching would require overwriting the user's uncommitted changes with the target branch's version.

Q5.3

Ans: Commits are created normally in the object store, and the HEAD file is updated with the new raw commit hash. However, because HEAD is not pointing to a branch file (like refs/heads/main), no branch reference is updated.

Recovery: If you switch to another branch, those "anonymous" commits are left orphaned because no branch points to them. A user can recover them by creating a new branch pointing directly to the orphaned commit's hash before switching away (e.g., pes branch <new-branch-name> <hash>). If they already switched away, they would need to find the hash in a reflog (if implemented) or by searching for unreachable commit objects.

Q6.1

Ans:

Mark: Start from all known reference tips (every file in refs/heads/, tags, and HEAD). Read the commit hashes. Recursively walk backward through the parent commits. For every commit visited, parse its root tree, and recursively traverse all sub-trees and blobs. Mark the hashes of all these visited objects.

Sweep: Iterate through every object file inside .pes/objects/. If the file's hash was not marked during the first phase, it is unreachable and can be safely deleted.

Data Structure: A Hash Set (typically implemented as a bitset or a hash table) is the most efficient way to track "reachable" hashes. Since SHA-256 hashes are large (32 bytes), storing prefixes (e.g., the first 8-12 bytes) in the set is often enough to guarantee uniqueness while saving memory.

Estimation: You would need to visit all 100,000 commits. Since branches just point to commits within that history, the 50 branches don't multiply the workload (they just provide starting points). Furthermore, you must visit every tree and blob referenced by those commits. If you track visited objects efficiently, you visit each unique object exactly once. Depending on repo churn, this means visiting hundreds of thousands to millions of objects.

Q6.2

Ans:

Why it's dangerous: A commit operation takes time. It writes blobs and trees to the object store before the final commit object is created and the branch ref is updated. If GC runs during this window, those newly written objects appear "unreachable" because no reference points to them yet.

The Race Condition:

Process A stages a file, writing a new blob X to the object store.

Process B (GC) starts its Mark phase. It traces from refs/, doesn't see blob X (because Process A hasn't made the commit yet), and marks X for deletion.

Process B deletes blob X.

Process A finishes creating the commit referencing X and updates the branch.

Result: A corrupt repository where a branch points to a commit that is missing its blobs.

How Git avoids this: Git relies on file modification timestamps (mtime). During GC, it does not delete all unreachable objects. It only deletes unreachable objects that are older than a specific grace period (usually 2 weeks). When Process A creates blob X, it has a very recent timestamp, so even though GC sees it as unreachable, it spares it from deletion.